STACK USING ARRAY - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>
#define MAX 5

int stack[MAX], top = -1;

void push()
{
    int x;
    if (top == MAX - 1)
    {
        printf("Overflow\n");
        return;
    }

    printf("Enter value: ");
    scanf("%d", &x);
    stack[++top] = x;
    printf("%d pushed to stack\n", x);
}

void pop()
{
    if (top == -1)
    {
        printf("Underflow\n");
        return;
    }

    printf("%d popped\n", stack[top--]);
}

void display()
{
    int i;
    if (top == -1)
    {
        printf("Empty stack\n");
        return;
    }

    printf("stack elements:\n");
    for (i = top; i >= 0; i--)
        printf("%d\n", stack[i]);
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1.Push\n2.Pop\n3.Display\n4.Exit\n\nChoose operation: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: display(); break;
            case 4: printf("Exiting...\n"); return 0;
            default: printf("Invalid choice\n");
        }
    }
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
1.Push
2.Pop
3.Display
4.Exit

Choose operation: Enter value: 10 pushed to stack

1.Push
2.Pop
3.Display
4.Exit

Choose operation: Enter value: 20 pushed to stack

1.Push
2.Pop
3.Display
4.Exit

Choose operation: stack elements:
20
10

1.Push
2.Pop
3.Display
4.Exit

Choose operation: 20 popped

1.Push
2.Pop
3.Display
4.Exit

Choose operation: stack elements:
10

1.Push
2.Pop
3.Display
4.Exit

Choose operation: Exiting...
```

CIRCULAR QUEUE USING ARRAY - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void insert()
{
    int x;
    if (front == (rear + 1) % MAX)
    {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter number to insert: ");
    scanf("%d", &x);

    if (front == -1)
        front = rear = 0;
    else
        rear = (rear + 1) % MAX;

    queue[rear] = x;
    printf("%d Inserted\n", x);
}

void delete()
{
    if (front == -1)
    {
        printf("Queue Underflow\n");
        return;
    }

    printf("%d deleted\n", queue[front]);

    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

void display()
{
    int i;
    if (front == -1)
    {
        printf("Queue is Empty\n");
        return;
    }

    printf("Queue: ");
    for (i = front; ; i = (i + 1) % MAX)
    {
        printf("%d ", queue[i]);
        if (i == rear)
            break;
    }
    printf("\n");
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Insert\n2. Delete\n3. Display\n4. Exit\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: insert(); break;
            case 2: delete(); break;
            case 3: display(); break;
            case 4: return 0;
            default: printf("Invalid Choice\n\n");
        }
    }
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: Enter number to insert: 10 Inserted

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: Enter number to insert: 20 Inserted

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: Queue: 10 20

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 10 deleted

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: Enter number to insert: 30 Inserted

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: Queue: 20 30

1. Insert
2. Delete
3. Display
4. Exit
Enter choice:
```

INFIX TO POSTFIX CONVERSION - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char ch)
{
    stack[++top] = ch;
}

char pop()
{
    return stack[top--];
}

char peek()
{
    return stack[top];
}

int precedence(char ch)
{
    if (ch == '+' || ch == '-')
        return 1;

    if (ch == '*' || ch == '/')
        return 2;

    if (ch == '^')
        return 3;

    return 0;
}

void infixToPostfix(char infix[], char postfix[])
{
    int i = 0, j = 0;
    char ch;

    while (infix[i] != '\0')
    {
        ch = infix[i];

        /* If operand, add to postfix */
        if (isalnum(ch))
        {
            postfix[j++] = ch;
        }

        /* If left parenthesis, push into stack */
        else if (ch == '(')
        {
            push(ch);
        }

        /* If right parenthesis */
        else if (ch == ')')
        {
            while (top != -1 && peek() != '(')
            {
                postfix[j++] = pop();
            }

            if (top != -1 && peek() == '(')
                pop();
        }

        /* If operator */
        else if (ch == '+' || ch == '-' ||
                 ch == '*' || ch == '/' || ch == '^')
        {
            while (top != -1 &&
                   peek() != '(' &&
                   (precedence(peek()) > precedence(ch) ||
                    (precedence(peek()) == precedence(ch) && ch != '^')))
            {
                postfix[j++] = pop();
            }

            push(ch);
        }

        i++;
    }

    /* Pop remaining operators */
    while (top != -1)
    {
        postfix[j++] = pop();
    }

    postfix[j] = '\0';
}

int main()
{
    char infix[MAX], postfix[MAX];

    printf("Enter an infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);

    return 0;
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Enter an infix expression: Postfix expression: AB+CDE^^*
```

LINEAR QUEUE USING ARRAY - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue()
{
    int x;
    if (rear == SIZE - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter element: ");
    scanf("%d", &x);

    if (front == -1)
        front = 0;

    queue[++rear] = x;
    printf("%d inserted into queue\n", x);
}

void dequeue()
{
    if (front == -1)
    {
        printf("Queue Underflow\n");
        return;
    }

    printf("%d deleted from queue\n", queue[front++]);

    if (front > rear)
        front = rear = -1;
}

void display()
{
    int i;
    if (front == -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: printf("Exiting...\n"); return 0;
            default: printf("Invalid choice\n");
        }
    }
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Enter element: 10 inserted into queue

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Enter element: 20 inserted into queue

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Queue: 10 20

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 10 deleted from queue

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Queue: 20

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Exiting...
```

POLYNOMIAL ADDITION USING ARRAYS - C PROGRAM

1. PROGRAM CODE

```
#include <stdio.h>
int main()
{
    int n1, n2, i=0, j=0, k=0, coeff1[20], coeff2[20], exp1[20], exp2[20], coeffsum[40], expsum[40];
    printf("Enter number of terms in first polynomial: ");
    scanf("%d", &n1);
    printf("Enter coefficient and exponent:\n");
    for(i = 0; i < n1; i++)
        scanf("%d %d", &coeff1[i], &exp1[i]);
    printf("Enter number of terms in second polynomial: ");
    scanf("%d", &n2);
    printf("Enter coefficient and exponent:\n");
    for(i = 0; i < n2; i++)
        scanf("%d %d", &coeff2[i], &exp2[i]);
    i = j = k = 0;
    while(i < n1 && j < n2)
    {
        if(exp1[i] > exp2[j])
        {
            coeffsum[k] = coeff1[i];
            expsum[k] = exp1[i];
            i++;
            k++;
        }
        else if(exp1[i] < exp2[j])
        {
            coeffsum[k] = coeff2[j];
            expsum[k] = exp2[j];
            j++;
            k++;
        }
        else
        {
            coeffsum[k] = coeff1[i] + coeff2[j];
            expsum[k] = exp1[i];
            j++;
            k++;
        }
    }
    while(i < n1)
    {
        coeffsum[k] = coeff1[i];
        expsum[k] = exp1[i];
        i++;
        k++;
    }
    while(j < n2)
    {
        coeffsum[k] = coeff2[j];
        expsum[k] = exp2[j];
        j++;
        k++;
    }
    printf("\nSum of polynomials:\n");
    for(i = 0; i < k; i++)
    {
        printf("%dx^%d", coeffsum[i], expsum[i]);
        if(i < k - 1)
            printf(" + ");
    }
    printf("\n");
    return 0;
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Enter number of terms in first polynomial: Enter coefficient and
exponent:
Enter number of terms in second polynomial: Enter coefficient and
exponent:

Sum of polynomials:
8x^3 + 4x^2 + 2x^1 + 8x^0
```

BROWSER NAVIGATION - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct node
{
    char url[100];
    struct node *prev, *next;
};

struct node *current = NULL;

void visit(char url[])
{
    struct node *temp, *nextnode, *newnode;

    temp = current ? current->next : NULL;
    while (temp != NULL)
    {
        nextnode = temp->next;
        free(temp);
        temp = nextnode;
    }

    if (current != NULL)
        current->next = NULL;

    newnode = (struct node *)malloc(sizeof(struct node));
    if (newnode == NULL)
    {
        printf("Memory allocation failed!\n");
        return;
    }

    strcpy(newnode->url, url);
    newnode->prev = current;
    newnode->next = NULL;

    if (current != NULL)
        current->next = newnode;

    current = newnode;
    printf("Visited: %s\n", current->url);
}

void back()
{
    if (current != NULL && current->prev != NULL)
    {
        current = current->prev;
        printf("Back to: %s\n", current->url);
    }
    else
        printf("No previous page\n");
}

void forward()
{
    if (current != NULL && current->next != NULL)
    {
        current = current->next;
        printf("Forward to: %s\n", current->url);
    }
    else
        printf("No next page\n");
}

void display()
{
    if (current != NULL)
        printf("Current page: %s\n", current->url);
    else
        printf("No page visited.\n");
}

int main()
{
    int choice;
    char url[100];

    while (1)
    {
        printf("\nWeb Browser\n1. Visit new page\n2. Back\n3. Forward\n4. Display current page\n5. Exit\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter URL: ");
                scanf("%s", url);
                visit(url);
                break;
            case 2: back(); break;
            case 3: forward(); break;
            case 4: display(); break;
            case 5: printf("Exiting...\n"); return 0;
            default: printf("Invalid choice!\n");
        }
    }
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Web Browser
1. Visit new page
2. Back
3. Forward
4. Display current page
5. Exit
Enter your choice: Enter URL: Visited: google.com

Web Browser
1. Visit new page
2. Back
3. Forward
4. Display current page
5. Exit
Enter your choice: Enter URL: Visited: github.com

Web Browser
1. Visit new page
2. Back
3. Forward
4. Display current page
5. Exit
Enter your choice: Back to: google.com

Web Browser
1. Visit new page
2. Back
3. Forward
4. Display current page
5. Exit
Enter your choice: Forward to: github.com

Web Browser
1. Visit new page
2. Back
3. Forward
4. Display current page
5. Exit
Enter your choice: Current page: github.com

Web Browser
1. Visit new page
2. Back
3. Forward
4. Display current page
5. Exit
Enter your choice: Exiting...
```

POLYNOMIAL ADDITION USING LINKED LIST - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int coeff, exp;
    struct node *next;
};

struct node *newNode(int c, int e){
    struct node *p = (struct node *)malloc(sizeof(struct node));
    p->coeff = c;
    p->exp = e;
    p->next = NULL;
    return p;
}

struct node *createPolynomial(){
    struct node *head = NULL, *tail = NULL, *p;
    int n, i, c, e;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++){
        printf("Enter coefficient and exponent: ");
        scanf("%d %d", &c, &e);

        p = newNode(c, e);
        if (head == NULL)
            head = p;
        else
            tail->next = p;
        tail = p;
    }
    return head;
}

struct node *addPolynomial(struct node *p1, struct node *p2){
    struct node *head = NULL, *tail = NULL, *p;
    int c, e;

    while (p1 != NULL || p2 != NULL){
        if (p2 == NULL || (p1 != NULL && p1->exp > p2->exp)){
            c = p1->coeff; e = p1->exp;
            p1 = p1->next;
        }
        else if (p1 == NULL || p2->exp > p1->exp){
            c = p2->coeff; e = p2->exp;
            p2 = p2->next;
        }
        else{
            c = p1->coeff + p2->coeff;
            e = p1->exp;
            p1 = p1->next;
            p2 = p2->next;
        }

        if (c == 0)
            continue;

        p = newNode(c, e);
        if (head == NULL)
            head = p;
        else
            tail->next = p;
        tail = p;
    }
    return head;
}

void display(struct node *p){
    while (p != NULL){
        printf("%dx^%d", p->coeff, p->exp);
        if (p->next != NULL)
            printf(" + ");
        p = p->next;
    }
    printf("\n");
}

int main(){
    struct node *p1, *p2, *result;

    printf("Enter first polynomial (descending exponents)\n");
    p1 = createPolynomial();

    printf("\nEnter second polynomial (descending exponents)\n");
    p2 = createPolynomial();

    result = addPolynomial(p1, p2);

    printf("\nFirst Polynomial: "); display(p1);
    printf("Second Polynomial: "); display(p2);
    printf("Result: "); display(result);

    return 0;
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Enter first polynomial (descending exponents)
Enter number of terms: Enter coefficient and exponent: Enter
coefficient and exponent: Enter coefficient and exponent:
Enter second polynomial (descending exponents)
Enter number of terms: Enter coefficient and exponent: Enter
coefficient and exponent: Enter coefficient and exponent:
First Polynomial: 5x^3 + 2x^1 + 1x^0
Second Polynomial: 3x^3 + 4x^2 + 7x^0
Result: 8x^3 + 4x^2 + 2x^1 + 8x^0
```

QUICK SORT - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>

void quickSort(int A[], int low, int high)
{
    int i, j, pivot, temp;

    if (low >= high)
        return;

    i = low;
    j = high;
    pivot = A[(low + high) / 2];

    while (i <= j)
    {
        while (A[i] < pivot)
            i++;
        while (A[j] > pivot)
            j--;

        if (i <= j)
        {
            temp = A[i];
            A[i] = A[j];
            A[j] = temp;
            i++;
            j--;
        }
    }

    if (low < j)
        quickSort(A, low, j);
    if (i < high)
        quickSort(A, i, high);
}

int main()
{
    int A[100], n, i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &A[i]);
    }

    quickSort(A, 0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
    {
        printf("%d ", A[i]);
    }

    return 0;
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Enter the number of elements: Enter 5 elements:
Sorted array:
10 20 30 40 50
```

PREFIX AND POSTFIX TRAVERSAL - C PROGRAM

1. PROGRAM CODE

```
#include <stdio.h>

struct Node
{
    char data;
    struct Node *left, *right;
};

int main()
{
    char expr[4];
    struct Node root, left, right;

    printf("Enter a simple arithmetic expression (e.g., a+b):\n");
    scanf("%3s", expr);

    left.data = expr[0];
    left.left = left.right = NULL;

    right.data = expr[2];
    right.left = right.right = NULL;

    root.data = expr[1];
    root.left = &left;
    root.right = &right;

    printf("Prefix expression:\n");
    printf("%c%c%c\n", root.data, root.left->data, root.right->data);

    printf("Postfix expression:\n");
    printf("%c%c%c\n", root.left->data, root.right->data, root.data);

    return 0;
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Enter a simple arithmetic expression (e.g., a+b):
Prefix expression:
+ab
Postfix expression:
ab+
```

BINARY SEARCH - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>

int main()
{
    int arr[100], n, key, i;
    int low = 0, high, mid;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d sorted elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter the item to search: ");
    scanf("%d", &key);

    high = n - 1;

    while (low <= high)
    {
        mid = (low + high) / 2;

        if (arr[mid] == key)
        {
            printf("Item found at position: %d\n", mid + 1);
            printf("Index: %d\n", mid);
            return 0;
        }
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    printf("Item not found.\n");
    return 0;
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Enter the number of elements: Enter 7 sorted elements:
Enter the item to search: Item found at position: 5
Index: 4
```

DEPTH FIRST SEARCH (DFS) - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>

int adj[20][20], visited[20];
int n;

void DFS(int v)
{
    int j;
    visited[v] = 1;
    printf("%d ", v);
    for (j = 0; j < n; j++)
    {
        if (adj[v][j] == 1 && visited[j] == 0)
        {
            DFS(j);
        }
    }
}

int main()
{
    int i, j, start;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &adj[i][j]);
        }
    }
    for (i = 0; i < n; i++)
        visited[i] = 0;
    printf("Enter the starting vertex (0 to %d): ", n - 1);
    scanf("%d", &start);
    printf("DFS Traversal: ");
    DFS(start);
    return 0;
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Enter the number of vertices: Enter the adjacency matrix:
Enter the starting vertex (0 to 4): DFS Traversal: 0 1 3 2 4
```

SPELL CHECKER USING HASHING - C PROGRAM

1. PROGRAM CODE

```c
#include <stdio.h>
#include <string.h>

#define SIZE 100

int hash(char word[])
{
    int i, sum = 0;

    for (i = 0; word[i] != '\0'; i++)
        sum += word[i];

    return sum % SIZE;
}

int main()
{
    char *hashTable[SIZE] = {NULL};
    char *dictionary[] = {"apple", "banana", "grape", "orange", "melon"};
    char input[50];
    int i, index;

    for (i = 0; i < 5; i++)
        hashTable[hash(dictionary[i])] = dictionary[i];

    printf("Enter a word to check: ");
    scanf("%s", input);

    index = hash(input);

    if (hashTable[index] != NULL && strcmp(hashTable[index], input) == 0)
        printf("The word is spelled correctly.\n");
    else
    {
        printf("The word is not found in the dictionary.\n");
        printf("Suggestions:\n");

        for (i = 0; i < SIZE; i++)
        {
            if (hashTable[i] != NULL && hashTable[i][0] == input[0])
                printf("%s\n", hashTable[i]);
        }
    }

    return 0;
}
```

2. ACTUAL PROGRAM OUTPUT

Representative full run (captured stdout)

```
Enter a word to check: The word is not found in the dictionary.
Suggestions:
apple
```